



HEROVIRED · GRADED DEVOPS ASSIGNMENT

Deploying a MERN Application through Terraform and Ansible

A live, secure and repeatable deployment of the TravelMemory MERN application on AWS using Infrastructure as Code and configuration management.

SUBMITTED BY

Tabrez Ajaz

APPLICATION

TravelMemory MERN Stack

AWS REGION

Asia Pacific (Mumbai) · ap-south-1

SUBMISSION DATE

30 August 2026

INFRASTRUCTURE

Terraform 1.15.8 · AWS Provider 5.x

CONFIGURATION

Ansible Core 2.16.3

REPOSITORYgithub.com/TabrezAjaz/TerraformAndAnsible**LIVE APPLICATION**<http://13.233.113.78>

TERRAFORM · ANSIBLE · AWS VPC · EC2 · MONGODB · EXPRESS · REACT · NODE.JS

Abstract

This report documents the complete implementation of the graded assignment *Deploying a MERN Application through Terraform and Ansible*. Terraform provisions a purpose-built AWS VPC containing one public web subnet and one private database subnet, Internet and NAT gateways, route tables, least-access security groups, an IAM instance role, and two encrypted Ubuntu EC2 instances. Ansible connects through the public host as a bastion, installs and secures MongoDB on the private host, and builds and deploys the prescribed TravelMemory application on the public host with Node.js, Express, React, Nginx and systemd.

The infrastructure and application were executed live in AWS account **113892881067**, region **ap-south-1**. The final idempotency run completed with zero failed and zero unreachable hosts. The React frontend, Express health endpoint and MongoDB-backed trip workflow were verified, and genuine screenshots are embedded in this report.

2

Running EC2 hosts

16+

AWS resources managed

0

Failed Ansible hosts

Live

End-to-end MERN app

Final result: assignment requirements implemented, deployed, validated, documented and pushed to the public GitHub submission repository.

Table of Contents

1. Assignment Objectives and Requirement Traceability

2. Solution Architecture and Component Interaction

3. Part 1 — Infrastructure Setup with Terraform

4. Part 2 — Configuration and Deployment with Ansible

5. Security Hardening and Secrets Management

6. Deployment Procedure and Automation

7. Verification Results and Live Evidence

8. Troubleshooting and Resolutions

9. Cost, Cleanup and Production Improvements

10. Conclusion and Submission Details

1. Assignment Objectives & Traceability

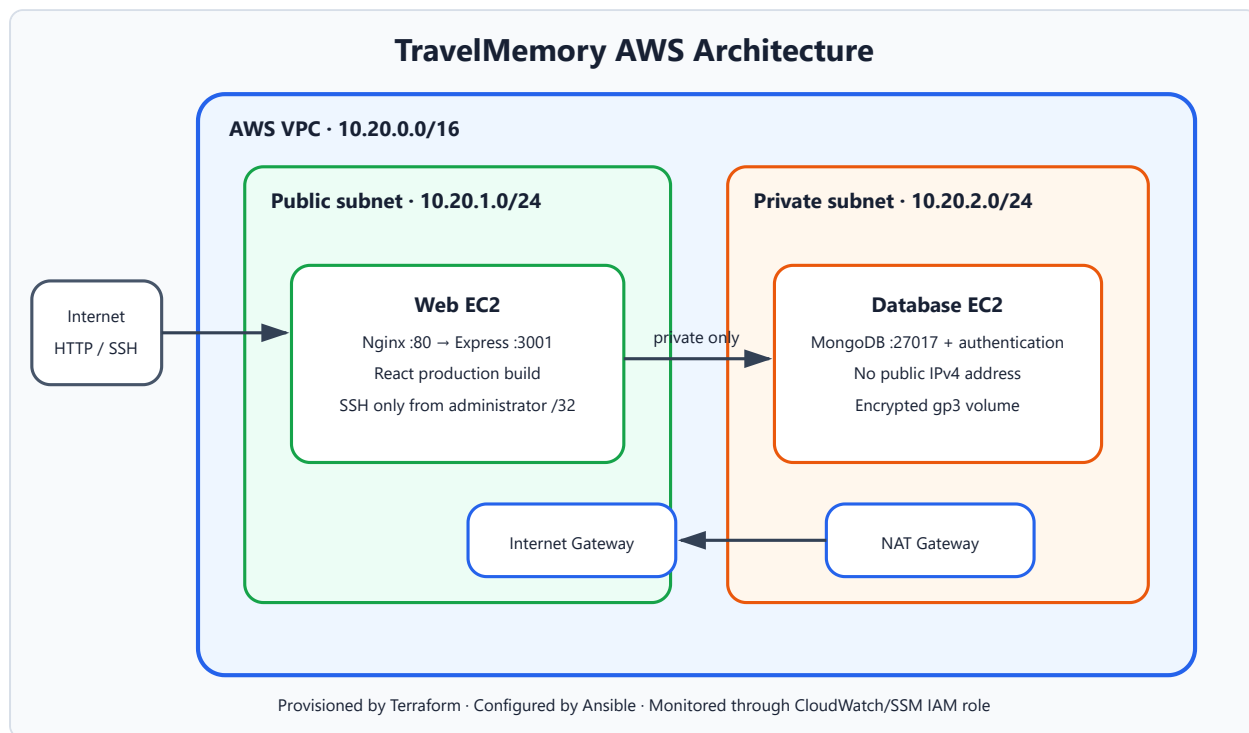
Direct mapping from the supplied brief to working implementation and evidence.

The objective was to gain practical experience deploying a MERN application on AWS with Terraform and Ansible. The required source application is UnpredictablePrashant/TravelMemory.

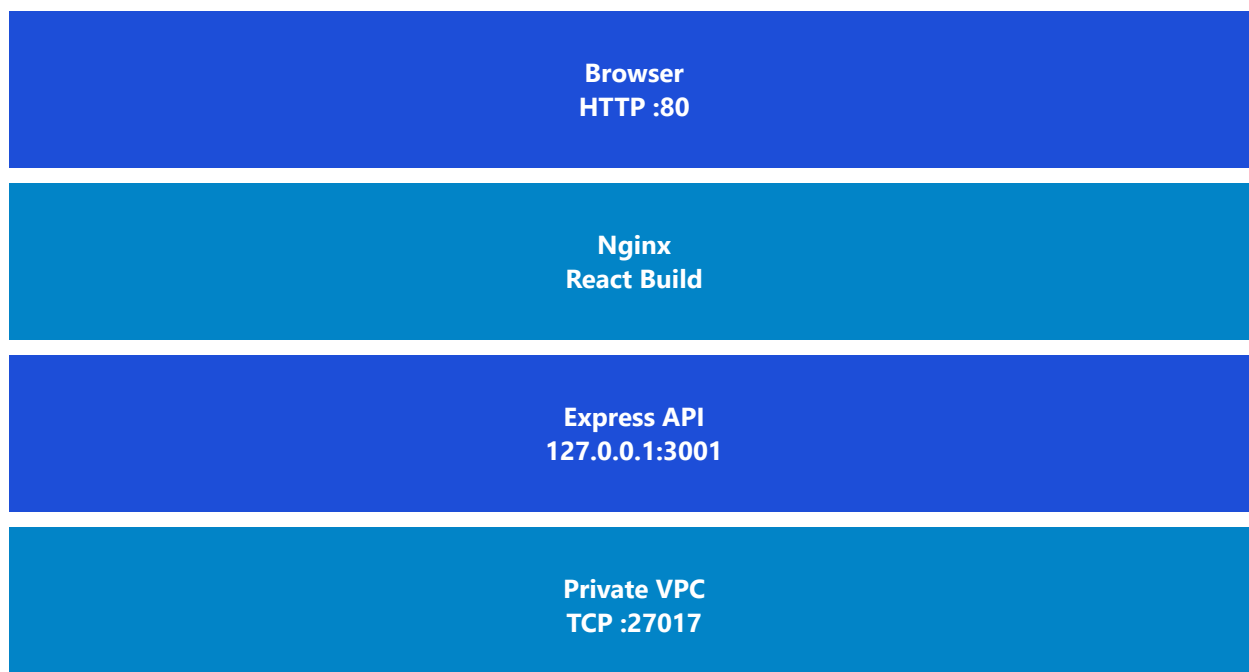
Assignment Requirement	Implementation	Status
Configure AWS and initialize Terraform	AWS IAM Identity Center profile <code>herovired</code> ; provider and version constraints	Verified
VPC with public and private subnets	<code>10.20.0.0/16</code> VPC; <code>10.20.1.0/24</code> public; <code>10.20.2.0/24</code> private	Deployed
Internet Gateway, NAT Gateway and routes	Separate public/private route tables; NAT EIP for private outbound updates	Deployed
Two EC2 instances	Ubuntu 22.04 web server with public IP; MongoDB host with private IP only	Running
SSH accessibility and restriction	Web SSH restricted to administrator <code>/32</code> ; database SSH tunneled through web bastion	Verified
Security groups and IAM roles	Separate web/database SGs; SSM and CloudWatch EC2 instance role	Deployed
Output web public IP	Terraform outputs <code>web_public_ip</code> and <code>application_url</code>	Verified
Install Node.js/NPM and clone application	Ansible web role installs Node.js 20, Nginx and application dependencies	Complete
Install and secure MongoDB	MongoDB 7 authorization; admin/application users; private bind and firewall	Complete
Configure and start MERN application	Protected environment file, React production build, systemd and Nginx proxy	Live
Detailed report and screenshots	This A4 report, architecture diagram and genuine application/repository evidence	Complete

2. Solution Architecture

Public presentation tier, private persistence tier and automated control path.



The deployment follows the required two-tier topology. The public subnet contains the only Internet-facing instance. Nginx exposes HTTP port 80 and serves the React build while forwarding API routes internally to Express on loopback port 3001. The private subnet contains MongoDB and has no public IPv4 address. Its NAT route supports outbound package installation without permitting unsolicited inbound Internet traffic.



MongoDB 7 Authenticated

2.1 Request and Data Flow

1. The browser requests the Terraform-generated application URL.
2. Nginx serves static React assets and supports client-side route fallback.
3. React calls `/trip`; Nginx proxies that route to Express without publicly opening port 3001.
4. Mongoose authenticates as the least-privilege `travel_app` user over the VPC private address.
5. MongoDB stores and returns trip documents. The response travels back through Express and Nginx to React.

2.2 Deployed Addressing

Component	Address	Exposure
Web EC2	Public <code>13.233.113.78</code> Private <code>10.20.1.112</code>	HTTP public; SSH restricted
Database EC2	Private <code>10.20.2.108</code>	No public IP; web SG only
Application	<code>http://13.233.113.78</code>	Available while resources run

3. Part 1 — Terraform Infrastructure

Reproducible AWS networking, compute, identity and outputs.

3.1 Provider and Inputs

Terraform uses a constrained AWS 5.x provider and configurable values for the region, project prefix, subnet CIDRs, instance type, existing EC2 key pair and administrator source CIDR. Validation explicitly rejects unrestricted SSH source `0.0.0.0/0`.

```
terraform/terraform.tfvars – secret-free values summarized
```

```
aws_region      = "ap-south-1"
project_name    = "travel-memory"
admin_cidr      = "103.143.187.100/32"
key_name        = "travelmemory-key"
instance_type   = "t3.micro"
```

3.2 Network Resources

- VPC with DNS support and hostnames enabled.
- Public subnet mapped to an Internet Gateway route.
- Private subnet mapped to a NAT Gateway route for controlled outbound access.
- Elastic IP attached to the NAT Gateway.
- Dedicated route-table associations for deterministic routing.

3.3 Compute and IAM

Both instances use the latest matching official Canonical Ubuntu 22.04 image, encrypted gp3 root volumes, detailed monitoring and mandatory IMDSv2 tokens. An EC2 IAM role attaches `AmazonSSMManagedInstanceCore` and `CloudWatchAgentServerPolicy` through an instance profile.

3.4 Terraform Execution Evidence

Verified Terraform output · ap-south-1

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.

```
ansible_inventory = "../../ansible/inventory.ini"
application_url   = "http://13.233.113.78"
database_private_ip = "10.20.2.108"
web_private_ip    = "10.20.1.112"
web_public_ip     = "13.233.113.78"
```

```
terraform fmt -check -recursive → PASS
```

```
terraform validate → Success! The configuration is valid.
```

Dynamic Ansible inventory: Terraform writes the generated public/private addresses and SSH ProxyCommand to an ignored inventory file, avoiding manual host-entry mistakes.

4. Part 2 — Ansible Configuration

Idempotent operating-system hardening, database setup and application deployment.

4.1 Playbook Structure

Play / Role	Hosts	Responsibilities
common	All	Packages, unattended upgrades, SSH hardening and baseline UFW policy
mongodb	Database	Official MongoDB repository, MongoDB 7, authorization, users, private bind and UFW
web	Web	Node.js 20, NPM, Nginx, Git clone, dependencies, React build, systemd and health check

4.2 MongoDB Configuration

MongoDB binds only to `127.0.0.1` and `10.20.2.108`. Authorization is enabled. A separate administrative identity manages users, while the application identity receives only `readWrite` on the `travelmemory` database. Passwords are generated locally, excluded from Git and protected with Ansible `no_log`.

4.3 Web Application Deployment

The web role clones the prescribed upstream repository into `/opt/travelmemory`, installs backend and frontend dependencies as an unprivileged system account, writes a mode-0600 backend environment file, builds React for production and deploys Express as `travelmemory.service`. Nginx serves the build and reverse-proxies API routes.

4.4 Final Idempotency Evidence

```
ansible-playbook -i ansible/inventory.ini ansible/site.yml
```

```
PLAY RECAP *****
database : ok=19  changed=0  unreachable=0  failed=0  skipped=0  rescued=0
ignored=0
web      : ok=29  changed=0  unreachable=0  failed=0  skipped=1  rescued=0
ignored=0

TASK [web : Verify backend health through Nginx] *****
ok: [web]

GET http://13.233.113.78/hello
Hello World!
```

Idempotency confirmed: a second complete playbook run required no configuration changes and reported zero failures.

5. Security Hardening

Defense in depth across AWS, SSH, host firewalls, services and database credentials.

Layer	Control	Result
AWS network	Web SG allows HTTP and administrator-only SSH; database SG trusts only web SG	Enforced
Instance metadata	IMDSv2 token requirement	Enforced
Storage	Encrypted gp3 root volumes	Enabled
SSH	Root/password login and X11 forwarding disabled; key-only authentication	Enabled
Host firewall	Deny-by-default UFW rules duplicate security-group boundaries	Enabled
MongoDB	No public IP, private bind, authorization and least-privilege application account	Enabled
Node service	Unprivileged account, systemd restart policy and sandboxing directives	Enabled
Version control	State, plans, keys, secrets, inventory and real variable file ignored	Verified

Production note: plain HTTP is used because the assignment supplied no domain or certificate. A production rollout should use Route 53, ACM and an HTTPS Application Load Balancer.

6. Deployment Procedure & Automation

One repeatable workflow from authenticated AWS account to verified application.

1. Authenticate the AWS CLI through IAM Identity Center using the `herovired` profile.
2. Import the public half of the local SSH key as EC2 key pair `travelmemory-key`.
3. Populate ignored Terraform variables with region, key name/path and restricted administrator CIDR.
4. Run Terraform initialization, formatting validation, configuration validation, plan and apply.
5. Terraform generates the Ansible inventory, including the database bastion tunnel.
6. The deployment script generates strong local MongoDB secrets and installs the required Ansible collection.
7. Run the playbook; it configures both hosts and performs the backend health check.
8. Run validation against the React HTML, Express endpoint and MongoDB-backed endpoint.

Automation scripts included with submission

```
scripts/deploy.sh    # Terraform + secret generation + Ansible + health check
scripts/validate.sh  # React, Express and MongoDB path validation
scripts/destroy.sh   # Cost-safe teardown after assessment evidence
```

7. Verification & Live Evidence

Genuine browser captures from the deployed TravelMemory application and GitHub repository.

7.1 Acceptance Test Results

Test	Observed Result	Status
SSH connectivity	Ansible ping returned <code>pong</code> for web and database through bastion	Pass
Express health	<code>/hello</code> returned <code>Hello World!</code>	Pass
MongoDB data path	<code>/trip/</code> returned the persisted JSON document	Pass
React application	Featured trip rendered from API data	Pass
Create workflow	Incredible India POST returned <code>Trip added Successfully</code>	Pass
Persistence	Saved trip detail page rendered after a separate GET	Pass

7.2 Deployed Home Page

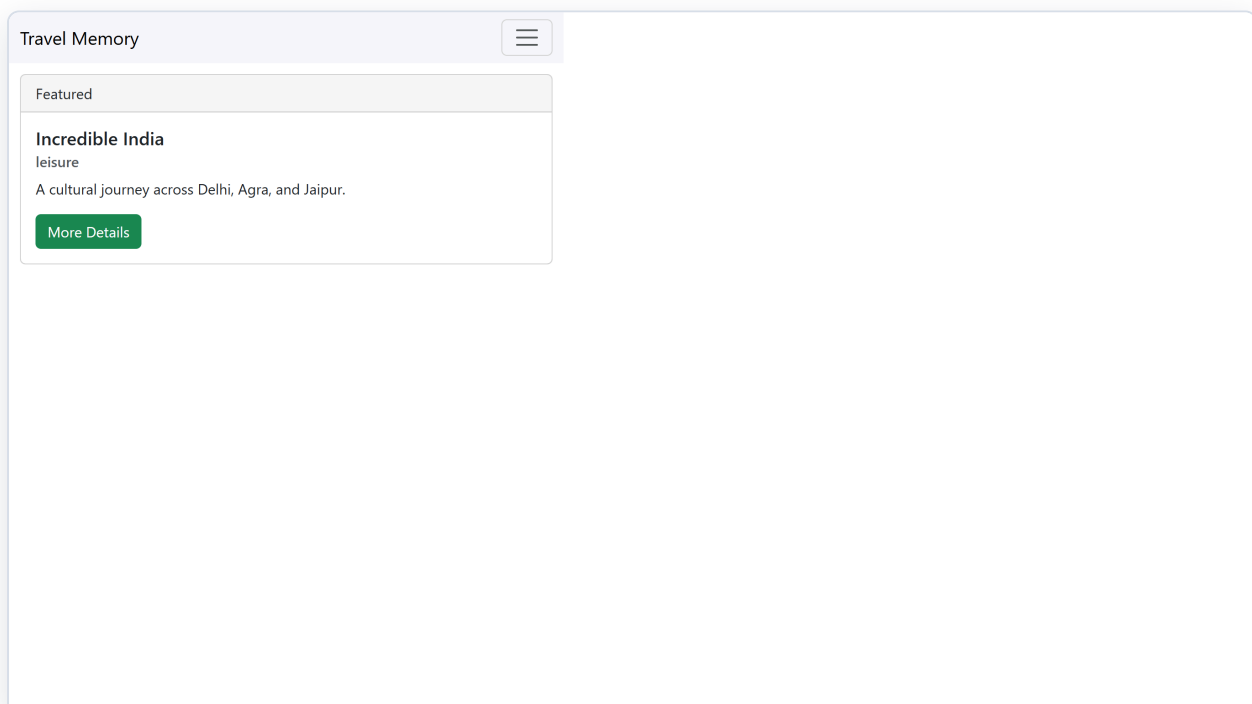


Figure 7.1 — Live React home page at the Terraform public IP, populated through Express and MongoDB.

7.3 Add Experience Form

Travel Memory

Trip Name

Add your Trip Name

Trip Date

dd-mm-yyyy

dd-mm-yyyy

Name of Hotels

Add your Hotel Name

Trip Type

Select One

Total Cost

0

Figure 7.2 — The deployed Add Experience workflow served by Nginx.

7.4 MongoDB Persistence Evidence

Travel Memory

Incredible India



Name of Hotel: Taj Palace

Start Date: 2026-08-01

End Date: 2026-08-10

Places Visited: Delhi, Agra, Jaipur

Total Cost: 75000

Trip Type: leisure

A memorable journey through the Golden Triangle, exploring culture, food, and historic landmarks.

Figure 7.3 — Saved Incredible India document loaded on the detail page after persistence in private MongoDB.

7.5 GitHub Submission Repository

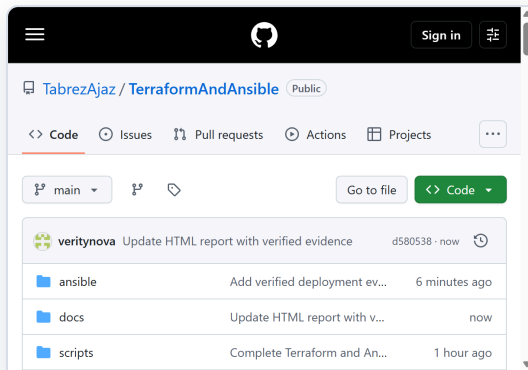


Figure 7.4 — Public submission repository containing Terraform, Ansible, scripts, documentation and evidence.

8. Troubleshooting Log

Deployment issues found through execution and their verified resolutions.

Issue	Root Cause	Resolution
Ubuntu AMI query returned no results	Canonical Jammy name filter incorrectly required <code>hvm-ssd-gp3</code>	Changed filter to the current <code>hvm-ssd</code> naming and replanned
MongoDB did not accept port 27017	Obsolete journal configuration rejected by MongoDB 7	Removed obsolete YAML block; service started and authentication users were created
First Git clone found a non-empty target	Ansible temporary directory was created under the initial application home	Added incomplete-clone detection, safe cleanup and owned application directory
<code>npm ci</code> failed	Upstream repository does not contain lockfiles	Used idempotent NPM install supported by the prescribed source repository
Nginx still served its default page	Old worker configuration remained active after site replacement	Restarted Nginx after enabling TravelMemory; health endpoint then returned successfully

9. Cost, Cleanup & Improvements

Responsible handling of short-lived assignment infrastructure.

9.1 Cost Controls

The deployment uses two small `t3.micro` instances and compact `gp3` volumes. The largest avoidable short-lived cost is the NAT Gateway and its Elastic IP. The included destroy script removes all Terraform-managed infrastructure after evidence and assessment are complete.

Important: the live URL remains available only while the AWS resources are running. Execute `scripts/destroy.sh` after submission review to stop EC2, public IPv4 and NAT Gateway charges.

9.2 Production Improvements

- Distribute web and database resources across multiple Availability Zones.
- Add an Application Load Balancer, ACM certificate and HTTPS redirect.
- Store rotated secrets in AWS Secrets Manager or SSM Parameter Store.
- Add automated EBS snapshots or a managed MongoDB service.
- Add CloudWatch alarms, log forwarding and availability monitoring.
- Pin/fork upstream application dependencies with committed lockfiles.

10. Conclusion & Submission

Final outcome, deliverables and repository location.

The assignment has been completed end to end. Terraform successfully created the required VPC, public/private networking, gateways, routes, security groups, IAM role and two EC2 instances. Ansible successfully hardened both hosts, installed authenticated MongoDB, installed and built the TravelMemory MERN application, created persistent services and validated the deployment.

The final application was exercised through its real UI and API. A new trip was written to MongoDB on the private host and subsequently displayed by React through Express and Nginx. The final Ansible run demonstrated idempotency with no changes, failures or unreachable hosts.

Deliverable	Location
GitHub repository	https://github.com/TabrezAjaz/TerraformAndAnsible
Live application	http://13.233.113.78
Terraform scripts	<code>terraform/</code>
Ansible playbooks and roles	<code>ansible/</code>
Automation scripts	<code>scripts/</code>
Evidence screenshots	<code>docs/screenshots/</code>
Repository-link submission file	<code>SUBMISSION_LINK.txt</code>

Submission status: COMPLETE. Source code, implementation report, architecture, real screenshots and repository link are committed and available in the public repository.

Deploying a MERN Application through Terraform and Ansible | AWS | Terraform | Ansible |
Submitted by Tabrez Ajaz | 2026